

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 8 of 22 — Acquisition: Core Satellite Extraction extract_satellite_data.py, extract_control_satellite_data.py — Poori Line By Line

Sakshi ke liye, Sakshi ki apni research par. PDF 6 mein humne coordinates dhunde (geocoding), PDF 7 mein humne control villages chune. Ab, is PDF mein, hum wahi asli kaam dekhenge jiske liye ye poora acquisition-pipeline bana tha — un coordinates par jaake, satellite se, asli data khinchna.

Ye do files, Google Earth Engine se, har gaon ke liye NDBI (built-up-area proxy) aur VIIRS night-lights nikaalti hain — treated villages ke liye ek file, control villages ke liye ek doosri. Yahi raw numbers hain jo aage PDF 11 (H1 test) aur PDF 12 (H4 DiD test) mein statistically test honge. Aur is file ki history mein ek poora, dramatic reversal chhupa hai — jahan ek single resume-logic bug fix karne se, poore project ka 'sabse strong result' ulta ho gaya.

Shuru Karte Hain — Ye Do Files Kya Karti Hain

Ab tak ka safar yaad karo: PDF 6 mein humne har gaon ka naam, coordinates mein badla. PDF 7 mein humne, unhi coordinates ke aas-paas, non-VVP control villages chune. Lekin ek coordinate, apne aap mein, kuch nahi batata — wo sirf ek jagah hai. Is PDF ki do files, us jagah par jaake, asli satellite-evidence nikaalti hain: kya wahan pehle se zyada 'built-up' (concrete, sadkein, buildings) dikh raha hai? Kya raat ko wahan pehle se zyada roshni hai (matlab, zyada bijli, zyada activity)? Ye do sawaal hi is poori research ka core hain.

EXAMPLE — Doctor Ki Checkup Analogy

Socho tu ek doctor hai, aur tujhe janna hai ki ek patient ka health improve hua hai ya nahi, do check-up dates ke beech. Sirf 'mujhe lagta hai behtar hai' bolna kaafi nahi — tujhe actual measurements chahiye: blood pressure, weight, sugar level, pehle bhi aur ab bhi. Ye do files exactly yahi kaam karti hain, satellite ke through — har gaon ka 'before' measurement (2021) aur 'after' measurement (2025) nikaalti hain, do independent 'vitals' ke liye: NDBI (built-up-area ka proxy, jaise weight) aur night-lights (activity ka proxy, jaise pulse-rate). Bina in measurements ke, koi bhi 'improvement hui ya nahi' ka daava sirf guesswork hota.

Do files kyun, ek nahi? `extract_satellite_data.py` treated (VVP-I) villages ke liye chalti hai — 258 villages (251 core-sample + 7 Himachal illustrative). `extract_control_satellite_data.py` control (non-VVP) villages ke liye — jo PDF 7 mein select hui thi. Dono files ka core-logic, jaisa hum aage dekhenge, bilkul same hai — sirf konsi village-list use hoti hai, aur resume kaise hota hai, wahan farak hai. File ka apna docstring hi ye batata hai: 'kept separate so the treated-sample run stays untouched' — matlab, agar control-extraction mein kabhi kuch galat ho, treated-side ka already-verified data accidentally corrupt na ho.

History Ek Nazar Mein — Ek Bug Jisne Poore Paper Ka Headline Palat Diya

Is PDF ki sabse badi kahani Development Log Entry 22 mein hai. Ek chhota, technical resume-logic bug — jisme purane checkpoint files ko sahi tarike se match nahi kiya ja raha tha — chhupa raha, tab tak jab tak ek poori, fresh re-extraction nahi hui. Jab hui, toh jo results pehle 'significant, surviving' lag rahe the, wo sab null ho gaye — aur ek naya result (jo pehle null tha) significant ban gaya. Hum ye poori kahani, code ke exact us hisse ke saath, aage dekhenge jahan ye bug tha.

Part A — extract_satellite_data.py: Treated Villages Ke Liye

Docstring

```
"""Buffers each village 500m, pulls NDBI (Sentinel-2) and VIIRS night-lights
for full_year/summer windows. Null (not zero) when a period has no
cloud-free images.
"""
```

Purpose: File ka apna, top-level introduction. Teen cheezein ek saath batayi ja rahi hain: kya extract hota hai (NDBI aur VIIRS), kaise (har gaon ke around ek 500-meter buffer), aur ek zaroori design-decision — agar kisi period mein koi cloud-free image nahi mili, result *null* hoga, *zero* nahi. Ye farak chhota nahi hai: zero ka matlab hota 'measured value zero thi', jabki null ka matlab hai 'measure hi nahi ho paya' — agar in dono ko mix kar diya jaaye, koi bhi statistical test galat results dega. Syntax: Triple-quoted module docstring, jaisa PDF 6/7 ki files mein dekha.

EXAMPLE — Ye File Kabhi Repo Mein Thi Hi Nahi — Entry 9 Ki Discovery

Development Log Entry 4 mein is file ka poora design likha gaya tha — buffer logic, NDBI formula, sab kuch. Lekin Entry 9 mein, ek submission-readiness audit ke dauraan, ek ajeeb gap mila: ye exact script, jisme har downstream analysis depend karti hai, kabhi src/acquisition/ folder mein commit hi nahi hui thi — sirf uska output (CSV files) the, source code nahi. Entry 4 mein likhi gayi logic se match karte hue, ise phir se banaya gaya, taaki poori pipeline source-se-reproducible ho, sirf cached outputs se nahi. Ye ek achha reminder hai: agar sirf output file hai, code nahi, toh research reproducible nahi hai — kisi ko bhi pata nahi chalega *kaise* wo number aaya.

Imports Aur Setup

```
import argparse
import os
import time

import ee
import pandas as pd
from dotenv import load_dotenv

load_dotenv()
```

Purpose: argparse command-line se options (jaise --window) padhne ke liye. os environment variables aur file-existence check ke liye. time wait/delay ke liye. ee — Google Earth Engine ki Python library, is poori file ka core tool; ye satellite imagery ko query aur process karne deti hai bina khud download kiye (processing Google ke servers par hoti hai, hum sirf result mangte hain). pandas table operations. dotenv se

`load_dotenv` — secret credentials `.env` file se load karne ke liye, jaisa PDF 6 mein Bhuvan token ke saath dekha tha. `load_dotenv()` ko turant call karna, file ke top par, taaki neechे jo bhi environment variables padhe jaayein, wo already load ho chuke hon. Syntax: Standard imports, jaisa pehle dekha. `load_dotenv()` — function ko call karna (parentheses ke saath), na ki sirf reference rakhna.

Path, Buffer, Aur Collection Constants

```
VILLAGES_PATH = "data/processed/border_optics_master_villages.csv"
BUFFER_RADIUS_M = 500

S2_COLLECTION = "COPERNICUS/S2_SR_HARMONIZED"
VIIRS_COLLECTION = "NOAA/VIIRS/DNB/MONTHLY_V1/VCMSLCFG"
```

Purpose: `VILLAGES_PATH` wahi `merge_geocoded.py` (PDF 6) ka output hai — 258 geocoded treated villages. `BUFFER_RADIUS_M = 500` har gaon ke point-coordinate ke around ek 500-meter radius ka circle define karta hai — is poore circle ke andar ka satellite-data average hoga, ek single point ki jagah. `S2_COLLECTION` Google Earth Engine mein Sentinel-2 satellite ki 'Surface Reflectance, Harmonized' image-collection ka poora naam hai — ye NDBI ke liye source imagery hai. `VIIRS_COLLECTION` NOAA/VIIRS satellite ki monthly night-lights collection hai (`VCMSLCFG` = VIIRS Cloud Mask, Straylight-Corrected — ek processed, cleaner version). Syntax: Char string constants, module-level. Earth Engine collections ko unke poore, official path-jaise naam se refer kiya jaata hai — ye Google ke catalog mein har dataset ka unique identifier hai.

EXAMPLE — 500 Meter Ka Buffer Kyun — Na Zyada Chhota, Na Zyada Bada

Development Log Entry 4 mein iska reasoning saaf hai: buffer 'chhota enough to stay village-specific, bada enough to average out single-pixel noise.' Agar buffer bahut chhota hota (jaise 10 meter — ek hi pixel), toh ek galat ya noisy pixel poora result bigaad sakta tha. Agar bahut bada hota (jaise 5km), toh aas-paas ke doosre gaon ya jungle ya khet bhi mix ho jaate, aur result us ek gaon ka nahi rehta. 500 meter ek beech ka, sensible choice hai — Village ke core settlement area ko cover karta hai, bina zyada 'noise' ya zyada 'dilution' ke.

WINDOWS — Dono Compositing-Options Ek Jagah

```
WINDOWS = {
  "full_year": {
    "before": ("2021-01-01", "2022-01-01"),
    "after": ("2025-01-01", "2026-01-01"),
    "out_path": "data/processed/border_optics_village_results.csv",
  },
  "summer": {
    "before": ("2021-06-01", "2021-10-01"),
    "after": ("2025-06-01", "2025-10-01"),
    "out_path": "data/processed/border_optics_village_results_summer.csv",
  },
}
```

Purpose: Ye dictionary dono compositing-window options ko ek jagah define karta hai — 'full_year' (poora calendar year, 2021 aur 2025) aur 'summer' (sirf June-September, dono saal). Har option apna 'before' date-range, 'after' date-range, aur apna output-file-path rakhta hai — taaki dono windows ka data alag files mein save ho, kabhi mix na ho. **Syntax:** Ye ek *nested* dictionary hai — dictionary ke andar dictionary. Bahar wali keys ("full_year", "summer") window-naam hain; andar wali keys ("before", "after", "out_path") har window ki apni settings hain. ("2021-01-01", "2022-01-01") ek tuple hai, do date-strings ka.

EXAMPLE — Dono Windows Kyun Rakhi Gayi — Entry 4 Se Entry 5 Tak Ki Kahani

Ye poori WINDOWS dictionary khud, do generations ki history ka result hai. Development Log Entry 4 mein sirf full_year tha — 2021 (poora saal) vs 2025 (poora saal). Result: NDBI mein koi significant increase nahi mila ($p=1.000$), night-lights borderline ($p=0.050$). Lekin ek doubt uthaya gaya: high-altitude Himalayan terrain mein, poore-saal ka composite mahino ke barfee (snow-covered) images bhi shaamil karta hai — aur agar 2021 aur 2025 mein snow-vs-clear images ka ratio thoda alag ho (jo random hai, depend karta hai kaunse din cloud-free the), akela ye hi NDBI value shift kar sakta tha, bina zameen par kisi asli badlaav ke. Entry 5 mein isi doubt ko test karne ke liye 'summer' window add hua — June-September, jab zyada elevation range snow-free hoti hai. Dono windows aaj tak saath rakhe gaye hain, kyunki dono individually defensible hain, aur inke beech ka farak khud ek finding ban gaya — jo hum aage dekhenge.

init_ee() — Google Earth Engine Se Connect Hona

```
def init_ee():
    # need EE_PROJECT set in .env or Initialize() fails
    project = os.environ.get("EE_PROJECT")
    try:
        ee.Initialize(project=project)
    except Exception:
        ee.Authenticate()
        ee.Initialize(project=project)
```

Purpose: Ye function Earth Engine ke saath connection shuru karta hai — is file ka koi bhi satellite-query karne se pehle, ye chalna zaroori hai. Pehle environment se ek Google Cloud project-ID padha jaata hai. Fir seedha Initialize() try kiya jaata hai — agar pehle se authenticated session hai (browser ya machine par saved credentials), ye seedha kaam kar jaayega. Agar nahi (koi bhi Exception), ee.Authenticate() call hota hai — ye browser-based login flow shuru karta hai — aur uske baad dobara Initialize(). **Syntax:** os.environ.get("EE_PROJECT") environment variable padhta hai, na milne par None (crash nahi). try/except Exception: — yahan broad Exception jaan-bujh kar hai, kyunki 'not yet authenticated' kai alag tarike ki error de sakta hai, aur inn sabka jawaab same hai — authenticate karo aur dobara try karo. ee.Initialize(project=project) do baar call ho raha hai — ek try block mein, ek except block mein, authenticate hone ke baad.

EXAMPLE — EE_PROJECT Kyun Zaroori Hai — Entry 13 Ki Real-World Surprise

Ye function jaisa simple dikhta hai, uske peeche ek real, practical surprise hai. Development Log Entry 13 mein likha hai: jab control-group ki extraction pehli baar live chalayi gayi, pata chala ki earthengine-api library ne apna requirement badal diya tha — ab Initialize() ko ek explicit Google Cloud project chahiye hota hai, sirf ek bare browser-reauth kaafi nahi hai jaisa pehle hota tha. **Fix:** EE_PROJECT environment variable add ki gayi, python-dotenv ke through wired — bilkul BHUVAN_TOKEN jaisa hi pattern jo PDF 6 mein dekha tha. Ye ek achha reminder hai ki external APIs/libraries time ke saath apna behaviour badal sakti hain — jo code kal chal raha tha, aaj ek naya, unexpected requirement maang sakta hai, aur isse robustly handle karna (clear error ya fallback ke saath) achhi practice hai.

mask_s2_clouds() — Cloudy Pixels Ko Chhupana

```
def mask_s2_clouds(image):  
    # QA60 bitmask: bit 10 = opaque clouds, bit 11 = cirrus  
    qa = image.select("QA60")  
    cloud_bit_mask = 1 << 10  
    cirrus_bit_mask = 1 << 11  
    mask = qa.bitwiseAnd(cloud_bit_mask).eq(0).And(qa.bitwiseAnd(cirrus_bit_mask).eq(0))  
    return image.updateMask(mask).divide(10000)
```

Purpose: Ye function ek Sentinel-2 image ke andar se cloudy pixels ko 'mask' karta hai — matlab, unhe calculation mein shaamil hone se rokta hai, taaki cloud ki wajah se galat NDBI value na aaye. Sentinel-2 images ke saath ek special band aata hai, QA60 ('Quality Assessment'), jisme har pixel ke liye kuch specific bits set hote hain agar wahan cloud ya cirrus (patli, upar ki cloud) ho. Ye function un dono bits ko check karta hai, aur sirf un pixels ko rakhta hai jahan *dono* bits clear (0) hain — matlab, genuinely saaf, bina-cloud wale pixels. Aakhir mein, image ko 10000 se divide kiya jaata hai — Sentinel-2 apne reflectance values ko integers mein store karta hai (scaling ke liye), isse divide karke unhe wapas asli 0-1 range mein laaya jaata hai. Syntax: `image.select("QA60")` ek specific band nikaalta hai poori image se. `1 << 10` aur `1 << 11` bit-shift operators hain — `1 << 10` ka matlab hai '1 ko 10 positions left shift karo', jo binary mein sirf 10th bit ko 'on' karta hai (decimal mein 1024). Ye ek standard tareeka hai ek specific bit-position ko represent karne ka. `.bitwiseAnd(mask)` image ke har pixel-value ko us bit-mask se AND karta hai — result sirf tab non-zero hoga agar wo specific bit image mein set tha. `.eq(0)` check karta hai ki result exactly 0 hai (matlab, wo bit set nahi tha — cloud nahi hai). `.And(...)` do conditions ko jodta hai — dono true honi chahiye. `.updateMask(mask)` Earth Engine ka function hai jo un pixels ko 'invisible' (masked-out) kar deta hai jahan mask False hai — aage koi bhi calculation unhe ignore karegi. `.divide(10000)` poori image ko element-wise 10000 se divide karta hai.

EXAMPLE — Ye Sabse Purani Cloud-Masking Approach Hai — Aage Ek Behtar Version Aayega

Ye QA60-bitmask approach is poore project ka *original* cloud-masking method hai, Entry 4 se hi. Ye kaam karta hai, aur is poore study ke saare headline results isi se aaye hain. Lekin baad mein, ek aur, zyada refined method — SCL (Scene Classification Layer) — explore kiya gaya, jisme ek specific bug (snow ko cloud maan lena) bhi mila aur fix hua. Us poori kahani ka apna dedicated PDF hai — PDF 9, 'Cloud Masking.' Yahan hum sirf itna samajhte hain ki QA60 method kaise kaam karta hai; SCL method aur uska snow-bug PDF 9 mein poori detail mein aayega.

ndbi_for_period() — NDBI Nikaalna, Poore Region Se Average Karke

```
def ndbi_for_period(buffered_geom, start, end):
    collection = (
        ee.ImageCollection(S2_COLLECTION)
        .filterBounds(buffered_geom)
        .filterDate(start, end)
        .map(mask_s2_clouds)
    )
    count = collection.size().getInfo()
    if count == 0:
        return None, 0
```

Purpose: Ye function ek gaon ke buffer-area ke liye NDBI nikaalta hai, ek specific date-range ke liye. Pehle, Sentinel-2 ki poori collection ko teen filters se guzara jaata hai: sirf wo images jo is buffer-area ko cover karti hain (`filterBounds`), sirf wo jo is date-range ke andar hain (`filterDate`), aur har image par cloud-masking apply karna (`.map(mask_s2_clouds)`). Fir, kitni images bachi, ye count kiya jaata hai. Agar bilkul zero images hain (is buffer/date-range ke liye kuch usable nahi mila), function turant `None, 0` return kar deta hai — crash karne ke bajaye, ek clean 'no data' signal. **Syntax:** Method-chaining (`.filterBounds(...).filterDate(...).map(...)`) — har method apni khud ki, modified collection wapas bhejta hai, jisse agla method chain ho sake, jaisa PDF 7 mein Overpass query building mein dekha tha. `.map(mask_s2_clouds)` function-ko-argument-ki-tarah-pass karna hai — `mask_s2_clouds` har image par individually apply hoga, poori collection ke andar (Earth Engine ke server par, ek loop ki tarah, lekin humein khud loop likhne ki zaroorat nahi). `.size().getInfo()` — Earth Engine mein zyada tar operations 'lazy' hote hain (seedha calculate nahi hote, jab tak specifically na maanga jaaye); `.getInfo()` Google ke server se asli value wapas mangwata hai, is point par.

```
composite = collection.median()
ndbi_image = composite.normalizedDifference(["B11", "B8"]).rename("NDBI")

# needs ee.Geometry not ee.Feature -- call .geometry() first if buffering a Feature
stats = ndbi_image.reduceRegion(
    reducer=ee.Reducer.mean(),
    geometry=buffered_geom,
    scale=10,
    maxPixels=1e9,
).getInfo()

return stats.get("NDBI"), count
```

Purpose: Agar images bachi hain, unka *median composite* banaya jaata hai — matlab, har pixel-position ke liye, sab available images ke us pixel ki median value li jaati hai (average se zyada robust, kyunki median outliers se kam affected hota hai — jaise ek ghost-cloud jo QA60 ne pura nahi pakda). Fir NDBI calculate hota hai —

`normalizedDifference(["B11", "B8"])` ek standard formula compute karta hai: $(B11 - B8) / (B11 + B8)$, jahan B11 Sentinel-2 ka SWIR1 (Short-Wave Infrared) band hai aur B8 NIR (Near-Infrared) band. Ye exactly NDBI (Normalized Difference Built-up Index) ka formula hai — built-up/concrete surfaces SWIR mein zyada reflect karte hain NIR ke comparison mein, jabki vegetation ulta karta hai, isliye ye ratio 'kitna built-up hai' ka ek proxy ban jaata hai. Aakhir mein, `reduceRegion` is poore NDBI-image ko buffer-area ke andar ek single number mein average kar deta hai — 10-meter pixel-scale par (Sentinel-2 ka native resolution NDBI-relevant bands ke liye). Syntax: `collection.median()` ek naya, single composite-image banata hai poori collection se, pixel-by-pixel median lekar. `.normalizedDifference(["B11", "B8"])` Earth Engine ka built-in function hai jo $(band1 - band2) / (band1 + band2)$ compute karta hai — yahan `band1=B11`, `band2=B8`. `.rename("NDBI")` result-band ka naam badalta hai, taaki aage usse naam se refer kiya ja sake. `reduceRegion(reducer=..., geometry=..., scale=..., maxPixels=...)` — `reducer=ee.Reducer.mean()` batata hai 'area ke andar sab pixels ka average lo'; `scale=10` meters mein pixel-size hai jisme calculation ho; `maxPixels=1e9` ek safety-limit hai (agar area bahut bada ho jaaye galti se, Earth Engine crash na ho, error de). `.getInfo()` asli values wapas mangwata hai. `stats.get("NDBI")` result-dictionary se NDBI value nikaalta hai — safe access, agar kisi wajah se missing ho toh crash nahi.

EXAMPLE — Buffer Ko `.geometry()` Karna Kyun Zaroori Hai

Ye comment (# needs `ee.Geometry` not `ee.Feature`) ek real bug ki yaad dilata hai jo Development Log Entry 4 mein mila. Jab ek Feature (jisme geometry ke saath-saath metadata bhi hota hai, jaise ek gaon ka naam) ko buffer kiya jaata hai, Earth Engine ka `.buffer()` operation wapas ek doosra Feature deta hai, Geometry nahi — aur `reduceRegion` ko specifically ek Geometry chahiye hota hai, kuch aur nahi chalega. Fix simple tha: buffered feature par `.geometry()` call karke seedha uski geometry nikaal lo, tabhi `reduceRegion` ko do. Ye ek chhoti, lekin classic 'API expects exactly this type' ki galti thi — bahut common jab ek naye tool (jaise Earth Engine) ke saath pehli baar kaam karte hain.

lights_for_period() — Night-Lights Nikaalna

```
def lights_for_period(buffered_geom, start, end):
    collection =
    ee.ImageCollection(VIIRS_COLLECTION).filterBounds(buffered_geom).filterDate(start,
    end)
    count = collection.size().getInfo()
    if count == 0:
        return None, 0

    composite = collection.select("avg_rad").mean()
    stats = composite.reduceRegion(
        reducer=ee.Reducer.mean(),
        geometry=buffered_geom,
        scale=500,
        maxPixels=1e9,
    ).getInfo()

    return stats.get("avg_rad"), count
```

Purpose: Ye function `ndbi_for_period()` jaisa hi structure follow karta hai, lekin VIIRS night-lights ke liye — aur ismein cloud-masking ki zaroorat nahi hai (VIIRS monthly composite already cloud-cleaned hota hai Google ki taraf se, kyunki uska naam khud 'VCMSCFG' — VIIRS Cloud Mask — batata hai). Filtering wahi (bounds + date), zero-count par wahi null-safe return. Composite is baar `.median()` nahi, `.mean()` hai — VIIRS ke liye average zyada standard practice hai, kyunki night-lights mein extreme-outlier-jaisi cloud-contamination NDBI jitni common nahi hoti. `reduceRegion` wahi pattern, lekin `scale=500` — NDBI ke 10-meter ke bajaye — kyunki VIIRS satellite ka apna native resolution hi roughly 500 meter hai (Sentinel-2 se kaafi coarser); 10-meter scale par reduce karna koi extra detail nahi dega, sirf compute waste hoga.

Syntax: Almost identical syntax pattern jo `ndbi_for_period()` mein dekha — `.select("avg_rad")` ek specific band chunta hai (average radiance), `.mean()` composite banata hai, `reduceRegion` area-average nikaalta hai.

extract_window() — Poore Pipeline Ka Orchestrator, Part 1: Resume Setup

```
def extract_window(window_key, checkpoint_every=10):
    cfg = WINDOWS[window_key]
    before_start, before_end = cfg["before"]
    after_start, after_end = cfg["after"]
    out_path = cfg["out_path"]

    outcome_cols = [
        "ndbi_before", "ndbi_after", "ndbi_before_image_count", "ndbi_after_image_count",
        "lights_before", "lights_after", "lights_before_image_count",
        "lights_after_image_count",
    ]
```

Purpose: Ye function poora extraction-run chalata hai, ek window ("full_year" ya "summer") ke liye. `checkpoint_every=10` ek default parameter hai — har 10 villages ke baad progress save hogi, jab tak caller kuch aur na bataye. `cfg = WINDOWS[window_key]` us window ki settings nikaalta hai. Tuple-unpacking se `before/after` date-ranges alag variables mein aate hain. `outcome_cols` ek list hai un aath columns ki jo har gaon ke liye final result banayenge — char NDBI-related (`before/after` value + `before/after` image-count), char lights-related, same pattern. **Syntax:** `WINDOWS[window_key]` dictionary se ek value nikaalna, key ke through. `before_start, before_end = cfg["before"]` tuple-unpacking, jaisa pehle dekha. List literal, aath string-entries.

```
# resume from checkpoint csv, not the master list, or we lose all progress
if os.path.exists(out_path):
    villages = pd.read_csv(out_path)
    print(f"Resuming from existing checkpoint: {out_path}")
    if "latitude" not in villages.columns or "longitude" not in villages.columns:
        # old checkpoints don't have lat/lon at all -- pull them back in via village_id
        coords = pd.read_csv(VILLAGES_PATH)[["village_id", "latitude", "longitude"]]
        villages = villages.merge(coords, on="village_id", how="left")
        missing_coords = villages["latitude"].isna().sum()
        if missing_coords:
            print(f" WARNING: {missing_coords} row(s) in the checkpoint have no matching
            "
                  f"village_id in {VILLAGES_PATH} -- their coordinates are still missing.")
    else:
        villages = pd.read_csv(VILLAGES_PATH)
```

Purpose: Ye resume-logic ka pehla hissa hai, aur is exact block mein ek real bug fix chhupa hai. Comment khud bata raha hai core-idea: agar ek purani checkpoint file exist karti hai, usi se resume karo (master list se nahi), warna saara progress kho jaayega. Lekin ek subtlety hai: purani checkpoint files (jo is file ke per-row-loop design se pehle ki hain) mein `latitude/longitude` columns thi hi nahi — sirf naye-design ki checkpoints mein hain. Agar ye check na ho, aage jo code `row["latitude"]` use karta hai, crash ho jaata. Fix: agar `lat/lon` missing hai checkpoint mein, master village-list

se, `village_id` ke through, unhe wapas merge kar liya jaata hai. Agar koi row match na ho paaye (kabhi na ho, safety ke liye), ek warning print hoti hai — chup-chaap missing data reh jaane ke bajaye. Syntax: `os.path.exists(path)` file-existence check. `"latitude"` not in `villages.columns` or `"longitude"` not in `villages.columns` — or se do column-existence checks jode gaye. `pd.read_csv(path)[["village_id", "latitude", "longitude"]]` — CSV padhke, sirf teen specific columns select karna (double square-brackets). `villages.merge(coords, on="village_id", how="left")` — pandas ka SQL-jaisa join operation: `on="village_id"` batata hai kis column se match karna hai, `how="left"` ka matlab hai 'saari villages rows rakho, chahe coords mein match mile ya na mile' (agar na mile, us row ke lat/lon NaN ban jaayenge). `.isna().sum()` kitne rows mein missing values hain, ginta hai.

EXAMPLE — `KeyError: 'longitude'` — Entry 22 Ka Pehla Bug

Ye poora block, jaisa hum ab dekh rahe hain, Development Log Entry 22 ka direct result hai — is block se pehle, ye lat/lon-missing-check exist hi nahi karta tha. Jab full-re-extraction attempt hui (Sikkim ki summer-data-gap fix karne ke liye, jo hum aage dekhenge), resume-logic ne assume kiya ki checkpoint file mein hamesha lat/lon columns hongi. Lekin committed checkpoint (`border_optics_village_results_summer.csv`) is file ke current, per-row-loop design se *pehle* ki thi — usme `system:index/.geo` jaisi purani columns thi (ek earlier, Export.table-based version ke leftovers), lat/lon bilkul nahi. Resume karne ki koshish seedha `KeyError: 'longitude'` crash ban gayi, jaise hi koi row re-extraction ke liye aayi. Fix exactly wahi hai jo humne upar dekha — coordinates ko master-list se, `village_id` ke zariye, wapas merge karna. Verify karne ke baad (258/258 rows ne apne coordinates wapas paaye, zero missing), ye fix production mein chalaya gaya.

```

for col in outcome_cols:
    if col not in villages.columns:
        villages[col] = None

print(f'--- Extracting satellite data, window='{window_key}' ---')
print(f'Before: {before_start} to {before_end} | After: {after_start} to {after_end}')

```

Purpose: Ye ek safety-loop hai — jo bhi outcome-columns abhi tak dataframe mein nahi hain (jaise ek bilkul fresh run mein), unhe khaali (None) se bana diya jaata hai. Fir ek chhota status-print, taaki console dekhne wale ko pata chale kaunsa window aur kaunse date-ranges chal rahe hain. Syntax: `for col in outcome_cols: if col not in villages.columns: villages[col] = None` — ek simple loop jo missing columns ko default value se bhar deta hai. F-strings status messages ke liye.

Main Loop — Har Village Process Karna

```

for i, row in villages.iterrows():
    # only skip once every outcome + image-count col is filled in
    if all(pd.isna(row.get(c)) for c in outcome_cols):
        continue

    point = ee.Geometry.Point([row["longitude"], row["latitude"]])
    buffered_geom = point.buffer(BUFFER_RADIUS_M)

```

Purpose: Har row (gaon) ke liye: pehle check karo ki kya saare aath outcome-columns already filled hain — agar haan, ye gaon 'already done' hai, skip. Ye check jaan-bujh kar sirf value-columns nahi, image-count-columns bhi dekhta hai (comment isi ko flag karta hai) — kyunki ek row jisme value hai lekin count missing hai, adhoori maani jaani chahiye. Agar skip nahi hua, ek Earth Engine Point banaya jaata hai us gaon ke coordinates se, aur usko BUFFER_RADIUS_M (500m) se buffer kiya jaata hai. Syntax: `all(pd.isna(row.get(c)) for c in outcome_cols)` ek generator-expression `all()` ke andar — 'har column c ke liye, check karo row mein wo missing toh nahi, aur sabke sab True hone chahiye tabhi True milega.' `continue` is row ko skip karke agli par jaata hai. `ee.Geometry.Point([lon, lat])` — Earth Engine mein bhi order (longitude, latitude) hai, shapely jaisa hi (PDF 7 mein dekha tha). `point.buffer(500)` ek circular buffer-geometry banata hai.

```

ndbi_before, ndbi_before_n = ndbi_for_period(buffered_geom, before_start,
before_end)
ndbi_after, ndbi_after_n = ndbi_for_period(buffered_geom, after_start, after_end)
lights_before, lights_before_n = lights_for_period(buffered_geom, before_start,
before_end)
lights_after, lights_after_n = lights_for_period(buffered_geom, after_start, after_end)

villages.at[i, "ndbi_before"] = ndbi_before
villages.at[i, "ndbi_after"] = ndbi_after
villages.at[i, "ndbi_before_image_count"] = ndbi_before_n
villages.at[i, "ndbi_after_image_count"] = ndbi_after_n

```

```
villages.at[i, "lights_before"] = lights_before
villages.at[i, "lights_after"] = lights_after
villages.at[i, "lights_before_image_count"] = lights_before_n
villages.at[i, "lights_after_image_count"] = lights_after_n
```

Purpose: Ye is poore function ka core hai — humne upar jo char helper-functions dekhe (ndbi_for_period, lights_for_period), unhe char baar call kiya jaata hai: NDBI ke liye before-window aur after-window, lights ke liye before-window aur after-window. Har call ek value aur ek image-count, dono, tuple-unpacking se milte hain. Fir sab aath results, dataframe ki isi row mein, apne-apne columns mein likh diye jaate hain. **Syntax:** Char function-calls, har ek tuple-unpacking ke saath (value, count = function(...)). Aath .at[i, "col"] = value assignments — wahi single-cell-update pattern jo PDF 6/7 mein baar-baar dekha.

```
if (i + 1) % checkpoint_every == 0:
    villages.to_csv(out_path, index=False)
    print(f"{i + 1}/{len(villages)} villages processed...")

time.sleep(0.2)

villages.to_csv(out_path, index=False)

n_valid = villages.dropna(subset=["ndbi_before", "ndbi_after"]).shape[0]
print(f"Done. {n_valid}/{len(villages)} villages have valid before/after NDBI data.")
print(f"Saved to {out_path}")
```

Purpose: Har checkpoint_every (10) villages ke baad, ek checkpoint save hota hai — wahi interruption-safety-pattern jo humne baar-baar dekha hai. Har village ke baad ek chhota 0.2-second wait, Earth Engine ke server ko overload na karne ke liye. Loop khatam hone ke baad, ek final save, aur ek summary — kitni villages ke paas genuinely valid (missing-nahi) NDBI data hai, dono before aur after ke liye. **Syntax:** (i + 1) % checkpoint_every == 0 — modulo check, jaisa PDF 6 mein dekha. time.sleep(0.2) 0.2 second ruk jaata hai — float ho sakta hai, sirf integers nahi. villages.dropna(subset=[col1, col2]) un rows ko hata deta hai jinme diye gaye columns mein se koi bhi missing (NaN) ho. .shape[0] resulting dataframe ki row-count deta hai.

__main__ Block

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Extract Sentinel-2 NDBI and VIIRS
night-lights per village.")
    parser.add_argument(
        "--window", choices=["full_year", "summer"], required=True,
        help="Which compositing window to run -- see module docstring.",
    )
    args = parser.parse_args()
```

```
init_ee()  
extract_window(args.window)
```

Purpose: Jab ye file seedhe chalayi jaati hai command-line se, ek argument-parser banaya jaata hai jo --window flag maangta hai (jaise python extract_satellite_data.py --window summer) — sirf "full_year" ya "summer" allowed hain (choices=), aur ye zaroori hai (required=True, bina diye script chalegi hi nahi). Fir Earth Engine se connect hota hai, aur extraction shuru hoti hai us window ke liye. Syntax: argparse.ArgumentParser(description=...) ek parser-object banata hai, jo --help chalane par ye description bhi dikhata hai. .add_argument(...) ek naya command-line option define karta hai — choices=[...] allowed values limit karta hai, required=True ka matlab hai bina is flag ke script error dega. parser.parse_args() actual command-line arguments ko padhta hai aur ek object deta hai jiske attributes (args.window) se unhe access kiya ja sakta hai.

Part B — `extract_control_satellite_data.py`: Control Villages Ke Liye

Ab hum doosri file par aate hain. Iska docstring khud saaf keh deta hai: *'Same extraction logic as `extract_satellite_data.py`, for the non-VVP control group; kept separate so the treated-sample run stays untouched.'* Aur ye bilkul sach hai — `init_ee()`, `mask_s2_clouds()`, `ndbi_for_period()`, aur `lights_for_period()`, ye chaaron functions is file mein *character-by-character* wahi hain jo Part A mein humne poori detail mein dekhe — ek bhi line alag nahi. Isliye unhe dobara, wahi-wahi, yahan repeat nahi karenge — jo humne Part A mein seekha, wahi yahan bhi apply hota hai. Jo genuinely alag hai — aur jahan is file ki apni, alag history hai — wahi ab detail mein dekhenge.

Docstring Aur Kyun Do Alag Files

```
"""Same extraction logic as extract_satellite_data.py, for the non-VVP
control group; kept separate so the treated-sample run stays untouched.
"""
```

Purpose: Docstring khud file ka design-rationale bata deta hai: logic same hai, lekin file alag hai, taaki treated-sample ka already-extracted, verified data kabhi bhi control-side ke kisi experiment/bug se accidentally affect na ho. Ye ek achhi engineering practice hai — do datasets jo alag-alag reliability-requirements rakhte hain (treated data zyada baar reuse hota hai, aur control-side jyada baar regenerate hota hai, jaisa PDF 7 mein dekha), unhe code-level par bhi separate rakhna, taaki ek ki galti doosre tak na pahunche. Syntax: Triple-quoted docstring, ab tak ka sabse chhota is style mein — do lines.

VILLAGES_PATH — Sirf Ek Line Ka Farak, Lekin Bada Matlab

```
VILLAGES_PATH = "data/processed/border_optics_control_villages.csv"
```

Purpose: Yahi ek line hai jo poori file ki 'identity' badalti hai — Part A ki file treated villages padhti hai, ye file PDF 7 ki output (control villages) padhti hai. Baaki constants (`BUFFER_RADIUS_M`, `S2_COLLECTION`, `VIIRS_COLLECTION`, `WINDOWS` — sirf `out_path` values control-specific file-naam ke saath) sab identical hain Part A se. Syntax: Ek simple string-constant, jaisa Part A mein dekha, sirf value alag.

init_ee(), mask_s2_clouds(), ndbi_for_period(), lights_for_period() — Bilkul Wahi

Ye chaaron functions — Earth Engine se connect hona, QA60 cloud-masking, NDBI nikaalna, VIIRS lights nikaalna — is file mein bilkul wahi hain jo humne Part A mein poori purpose+syntax detail ke saath dekhe. Ek bhi character alag nahi. Agar tujhe koi ek line yaad nahi hai, Part A wapas palat ke dekh sakti hai — wahi explanation yahan bhi apply hoti hai, EE_PROJECT ki history se lekar .geometry() bug tak, sab.

extract_window() — Yahan Se Asli Farak Shuru Hota Hai

```
def extract_window(window_key, checkpoint_every=10):
    cfg = WINDOWS[window_key]
    before_start, before_end = cfg["before"]
    after_start, after_end = cfg["after"]
    out_path = cfg["out_path"]

    outcome_cols = [
        "ndbi_before", "ndbi_after", "ndbi_before_image_count", "ndbi_after_image_count",
        "lights_before", "lights_after", "lights_before_image_count",
        "lights_after_image_count",
    ]
```

Purpose: Ye setup Part A jaisa hi hai — window-config nikaalna, date-ranges unpack karna, outcome-columns ki list. Koi naya syntax nahi. Syntax: Wahi pattern jo Part A mein dekha.

```
# control village list gets fully regenerated each time (village_id isn't stable),
# so resume by matching lat/lon coords, not village_id
villages = pd.read_csv(VILLAGES_PATH)
for col in outcome_cols:
    if col not in villages.columns:
        villages[col] = None
```

Purpose: Yahan pehla genuine farak hai. Comment khud is file ka sabse important design-decision bata deta hai: control village-list *har baar poori tarah regenerate hoti hai* (PDF 7 mein dekha — 753→735→732→721, har regeneration par village_id fresh 1 se banta hai), isliye resume karne ke liye village_id par bharosa nahi kiya ja sakta — coordinate se match karna padta hai. Ye Part A se ekdum alag approach hai, jahan treated village-list stable rehti hai (isliye wahan village_id-merge surakshit tha). Syntax: villages = pd.read_csv(VILLAGES_PATH) — Part A mein ye sirf 'else' branch mein tha (jab checkpoint na ho); yahan ye hamesha, unconditionally chalta hai — control-list hamesha fresh padhi jaati hai, checkpoint se nahi. Baaki, column-filling loop wahi pattern.

EXAMPLE — village_id Kyun Bharosemand Nahi — Entry 15 Ki Asli Kahani

Ye poori design-choice — coordinate-based resume, village_id-based nahi — seedha Development Log Entry 15 se aati hai, aur ye *isi* file ke baare mein hai. Entry 15 mein

likha hai: jab control-village-list 753 se 735 par regenerate hui thi (PDF 7 ka Entry 14 dedup-fix), `extract_control_satellite_data.py` ka purana resume-logic sirf ye check karta tha ki kya ek given `village_id` ke liye NDBI/lights values already filled hain. Lekin `village_id` sirf ek sequential 1..N index hai, jo har regeneration par naye sire se banta hai. Purani 753-row checkpoint file abhi bhi disk par thi, `village_id` 1 se 753 tak values ke saath — toh naye 735-village list ki har ek row ko usi index par ek 'complete' match mil gaya, aur script seedha 'done' maan ke ek bhi naya Earth Engine call kiye bina skip ho gaya. Result: `did_model.py` ke numbers, byte-for-byte, purane bugged numbers nikle — aur yahi wajah thi ki ye pakda gaya (ye purani, still-committed 753-row file se locally reproduce ho gaye, terminal se paste kiye gaye kuch bhi trust karne se pehle). Fix: resume-check ko `village_id` ki jagah coordinate (rounded lat/lon) par match karne ke liye rewrite kiya gaya. Yahi fix hai jo humne PDF 7 mein bhi `select_control_villages.py` ke context mein dekha tha, aur yahi wo asli file hai jahan se ye lesson pehli baar seekha gaya.

Coordinate-Based Resume — Poora Logic, Line By Line

```
if os.path.exists(out_path):
    existing = pd.read_csv(out_path)
    current_coords = set(zip(villages["latitude"].round(6), villages["longitude"].round(6)))
    existing_coords = (
        set(zip(existing["latitude"].round(6), existing["longitude"].round(6)))
        if {"latitude", "longitude"}.issubset(existing.columns) else set()
    )
```

Purpose: Agar purani checkpoint file hai, use padha jaata hai. Fir do sets banaye jaate hain: `current_coords` — abhi ki (fresh-padhi) village-list ke saare coordinates, aur `existing_coords` — checkpoint file ke coordinates (agar usme lat/lon columns hain hi, warna khaali set — safety). In dono sets ko compare karke decide hoga ki resume karna hai ya fresh start. Syntax: `set(zip(col1.round(6), col2.round(6)))` — `.round(6)` ek pandas *Series* method hai jo poore column ko ek saath round karta hai. `zip(...)` do columns ko pair-pair jodta hai. `set(...)` sabko ek set mein daal deta hai — order-independent, duplicates-free comparison ke liye perfect. Conditional expression: `set(...)` if condition else `set()` — agar zaroori columns hain, real set banao, nahi toh khaali set (crash se bachne ke liye). `{"latitude", "longitude"}.issubset(existing.columns)` check karta hai ki dono naam `existing.columns` ke andar hain ya nahi.

```
if existing_coords and existing_coords == current_coords:
    # both sides gotta use the same .round(6) call (Series.round, not scalar round())
    # or floating point makes a few coords mismatch even though the sets match
overall
    lookup = existing.copy()
    lookup["_coord_key"] = list(zip(lookup["latitude"].round(6),
    lookup["longitude"].round(6)))
    lookup_dict = lookup.set_index("_coord_key")[outcome_cols].to_dict("index")
```

Purpose: Agar dono sets exactly match karte hain (matlab, checkpoint file *bilkul usi* village-list ke liye hai jo abhi padhi gayi), tabhi resume hota hai. Ek naya `_coord_key` column banaya jaata hai checkpoint mein, rounded (lat, lon) tuples ka. Fir ye poora checkpoint ek *lookup dictionary* mein badal diya jaata hai — coordinate se seedha uske outcome-values tak. Syntax: Comment mein ek bahut zaroori detail hai, jo aage ek real bug ki wajah banega — dhyan se padhna. `lookup = existing.copy()` independent copy. `list(zip(col1.round(6), col2.round(6)))` — pehle wahi coordinate-tuples banaye, ab ek naya column mein assign kiye. `lookup.set_index("_coord_key")` dataframe ka index badalta hai — ab har row apne coordinate-tuple se address ho sakti hai, uske sequential number se nahi. `[outcome_cols].to_dict("index")` — sirf outcome columns select karke, ek dictionary mein badal dena jiski keys index-values (coordinate-tuples) hain aur values har row ka poora data (khud ek dictionary ki tarah).

```

villages_keys = list(zip(villages["latitude"].round(6), villages["longitude"].round(6)))
missing_keys = 0
for i, key in zip(villages.index, villages_keys):
    values = lookup_dict.get(key)
    if values is None:
        missing_keys += 1
        continue
    for col in outcome_cols:
        villages.at[i, col] = values[col]
    print(f"Resuming from existing checkpoint: {out_path} (matched by coordinate,
{len(existing)} rows)")
    if missing_keys:
        print(f" NOTE: {missing_keys} row(s) had no exact coordinate match (rounding
edge case) -- will be re-extracted.")
    else:
        print(f" Checkpoint at {out_path} doesn't match current village list -- starting
fresh.")

```

Purpose: Ab har current village ke liye (unka apna coordinate-key banaake), lookup-dictionary mein us coordinate ko dhoonda jaata hai. Agar mila (values None nahi hai), uske saare outcome-values current dataframe mein copy kar diye jaate hain — resume ho gaya, bina Earth Engine call kiye. Agar nahi mila, ek counter badhta hai aur wo row khaali reh jaati hai (age re-extract hogi). Agar dono sets match hi nahi karte the (matlab genuinely alag village-list hai), koi resume nahi hota — poori list fresh se extract hogi, ek clear message ke saath. Syntax: `lookup_dict.get(key)` — safe dictionary access, na milne par None, crash nahi. `if values is None:` explicitly None check karna hai (is operator, `==` nahi — Python mein None ke liye is use karna convention hai). Nested loop: bahar wala har village ke liye, andar wala har outcome-column ke liye value copy karta hai.

EXAMPLE — KeyError: (27.903587, 96.940911) — Entry 22 Ka Doosra, Zyada Subtle Bug

Ye poora coordinate-matching design, jaisa hum abhi dekh rahe hain, Entry 15 ke baad se hi tha. Lekin Entry 22 mein, jab Sikkim ki summer-data-gap fix karne ke liye control-group bhi restart se re-extract kiya ja raha tha, is hi logic ke ek *purane* version mein ek naya, subtle bug mila: script crash ho gaya KeyError: (27.903587, 96.940911) ke saath — ye village 'Shirong' (Anjaw) ka coordinate tha, aur crash restart ke row 610/735 par hua, ek genuine, valid row par, corrupt data par nahi. Root cause: purani version mein, 'kya poora checkpoint match karta hai' wala aggregate check pandas ki vectorized Series.round(6) use karta tha, lekin har row ka apna, individual lookup Python ke built-in scalar round() se banta tha — aur ye dono rounding-paths, floating-point ki wajah se, 6th decimal par kabhi-kabhi *alag* answer de sakte hain, chahe underlying number asal mein wahi ho. Result: ek real, non-corrupt row, ek exact-match .at[] lookup mein fail ho gayi, aur poora run crash ho gaya. Fix, jo humne upar ke code mein dekha: dono taraf Series.round(6) consistently use karna (comment isi ko explicitly flag karta hai — 'both sides gotta use the same .round(6) call'), aur fragile .at[] lookup ko dict.get() se replace karna, jo gracefully degrade hota hai (missing-key mile toh us row ko re-extract kar do, crash mat karo) — ye exactly wahi lookup_dict.get(key) pattern hai jo humne abhi dekha. Ye ek bahut zaroori lesson hai: jab bhi ek hi value do alag jagah, do alag rounding-functions se round ki jaaye, unka match karna guaranteed nahi hai — hamesha same function, same jagah use karo.

Baaki Ka Main Loop — Bilkul Part A Jaisa

```
print(f"--- Extracting control-group satellite data, window='{window_key}' ---")
print(f"Before: {before_start} to {before_end} | After: {after_start} to {after_end}")
print(f"{len(villages)} control villages to process")
```

Purpose: Ek extra print line yahan hai jo Part A mein nahi thi — kitni control villages process hongy, upfront hi bata dena. Ye exactly wahi print hai jiske through Entry 37 mein pakda gaya tha ki do extraction scripts (extract_dynamicworld_built.py, extract_sar_backscatter.py) galti se purani 732-village checkpoint reuse kar rahi thi — unka print '732 villages to process' dikha raha tha, jabki current list mein 721 thi. Ek simple print-statement, lekin bahut useful debugging-signal. Syntax: Teen print(f"...") calls, status update ke liye.

```
for i, row in villages.iterrows():
    if all(pd.isna(row.get(c)) for c in outcome_cols):
        continue

    point = ee.Geometry.Point([row["longitude"], row["latitude"]])
    buffered_geom = point.buffer(BUFFER_RADIUS_M)

    ndbi_before, ndbi_before_n = ndbi_for_period(buffered_geom, before_start,
before_end)
```

```

    ndbi_after, ndbi_after_n = ndbi_for_period(buffered_geom, after_start, after_end)
    lights_before, lights_before_n = lights_for_period(buffered_geom, before_start,
before_end)
    lights_after, lights_after_n = lights_for_period(buffered_geom, after_start, after_end)

    villages.at[i, "ndbi_before"] = ndbi_before
    villages.at[i, "ndbi_after"] = ndbi_after
    villages.at[i, "ndbi_before_image_count"] = ndbi_before_n
    villages.at[i, "ndbi_after_image_count"] = ndbi_after_n
    villages.at[i, "lights_before"] = lights_before
    villages.at[i, "lights_after"] = lights_after
    villages.at[i, "lights_before_image_count"] = lights_before_n
    villages.at[i, "lights_after_image_count"] = lights_after_n

    if (i + 1) % checkpoint_every == 0:
        villages.to_csv(out_path, index=False)
        print(f" {i + 1}/{len(villages)} control villages processed...")

    time.sleep(0.2)

villages.to_csv(out_path, index=False)
n_valid = villages.dropna(subset=["ndbi_before", "ndbi_after"]).shape[0]
print(f"Done. {n_valid}/{len(villages)} control villages have valid before/after NDBI data.")
print(f"Saved to {out_path}")

```

Purpose: Ye poora block Part A ke main-loop se, syntax aur logic ke hisaab se, character-by-character identical hai (sirf print-messages mein 'control' shabd extra hai). Humne har line ka matlab Part A mein poori detail se dekha hai — completeness-check, point-aur-buffer banana, char extraction-calls, results likhna, checkpoint saving, final summary. Yahan dobara derive karne ki zaroorat nahi. **Syntax:** Wahi syntax jo Part A mein dekha.

__main__ Block

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Extract Sentinel-2 NDBI and VIIRS
night-lights for the control group.")
    parser.add_argument("--window", choices=["full_year", "summer"], required=True)
    args = parser.parse_args()

    init_ee()
    extract_window(args.window)

```

Purpose: Wahi structure jo Part A mein dekha, sirf description-string mein 'control group' add hua hai, aur --window argument ke saath is baar koi extra help= text nahi hai (Part A mein tha) — ek chhota, cosmetic farak, function same. **Syntax:** Wahi argparse pattern, jaisa Part A mein poori detail se dekha.

Ab, Poori Kahani — Entry 21 Se 22 Tak, Jisne Sab Kuch Palat Diya

Humne code ke andar-andar teen jagah is bug-fix ki jhalak dekhi — lat/lon backfill, coordinate-based resume, rounding-consistency fix. Ab chalo poori kahani, ek saath, samajhte hain — kyunki iska result is poore project ka sabse bada, sabse important moment hai.

Entry 21 — Ek Chhoti Si Doubt Jo Badi Nikli

Development Log Entry 21 mein, ek external review ka jawaab dete hue, ek gehri check ki gayi: kya summer-window mein Sikkim ka 'zero valid data' genuinely permanent fact hai, ya sirf ek purani extraction-run ka artifact? Comparison kiya gaya `extract_satellite_data.py` (do-point 2021/2025 extraction) aur `extract_multiyear_satellite_data.py` (teen-point 2021/2023/2025 extraction, jo baad mein alag se chalaayi gayi thi) ke beech — dono *exact same* query chalate hain summer window ke liye, 2021 aur 2025 dono saal, same collection, same cloud-mask, same buffer, same date-range. Lekin results alag nikle: do-point extraction mein saare 31 Sikkim villages ke `before_image_count = 0` aur `after_image_count = 0` the — zero images, NDBI null. Teen-point extraction mein, *wahi* 31 villages, *wahi* 2021/2025 date-ranges ke liye, `ndbi_n_2021 = 24` aur `ndbi_n_2025 = 31` dikhati thi — 24 aur 31 real scenes mile, koi bhi null nahi.

Ye koi code-bug nahi tha — dono scripts ki query-logic identical thi. Ye Sentinel-2 ke apne satellite-archive ka ek genuine, real-world behaviour tha: Earth Engine ka archive, do alag extraction-runs ke beech (jo mahino apart chalaayi gayi thin), *purane* dates ke liye bhi naye scenes backfill karta reh sakta hai. Jab pehli baar summer-extraction chali thi, us din tak jitne scenes archive mein the, unhi se result bana — baad mein jab archive mein aur scenes add hue (khaaskar Sikkim jaisi heavy-cloud region ke liye, jahan koi bhi extra clear-day scene bahut fark daalta hai), purana result stale ho gaya, bina kisi code ke badle.

Entry 22 — Poori, Fresh Re-Extraction, Aur Do Bugs Jo Raaste Mein Mile

Entry 21 ki finding ko action mein badalte hue, Entry 22 mein ek poori, same-day, dono groups (treated aur control) ki fresh re-extraction chalaayi gayi — taaki na sirf Sikkim ka data complete ho, balki treated aur control dono side ka data *ek hi din* ke archive-snapshot se aaye, taaki koi vintage-mismatch na ho. Isi re-extraction attempt ke dauraan, humne upar dekhe wo dono bugs pakde gaye — pehla `extract_satellite_data.py` ke resume-logic mein (lat/lon missing checkpoint), doosra `extract_control_satellite_data.py` mein (rounding-mismatch). Dono fix hue, verify hue,

aur fir poori extraction chali.

Extraction	Result
Treated (summer, full-year)	258/258 valid (251 core + 7 Himachal)
Control (summer, full-year)	735/735 valid
Vintage	Treated aur control, dono ek hi din (same-day) extract hue

Result — Poore Project Ka Sabse Bada Reversal

Jab summer-window ke saare tests is naye, complete, same-day data par dobara chalaaye gaye, jo bhi 'significant, surviving' tha, wo sab null ho gaya — aur ek naya result (jo pehle stable null tha) significant ban gaya.

Test	Pehle (stale/partial)	Baad Mein (fresh, complete)
H1 NDBI, treated-only	n=154, p<0.000001 (significant)	n=251, p=0.435831 (null)
H1 lights, treated-only	n=251, p=0.9999 (null)	n=251, p=0.999994 (null, same)
H4 NDBI, control DiD	coef=+0.0322, p=0.00033 (sig.)	coef=-0.0038, p=0.44701 (null)
H4 lights, control DiD	coef=+0.1413, p=0.0044 (sig.)	coef=+0.0374, p=0.36553 (null)
H3 NDBI border-proximity	rho=0.038, p=0.641 (null)	rho=0.291, p=2.6e-6 (sig., naya)
Buffer sweep (3 radii)	sab p<0.002 (sig.)	p=0.126/0.436/0.899 (null)
Leave-one-district-out	14/14 sig.	0/14 sig.
Baseline (2021) balance	p=0.00073/0.0165 (imbalanced)	p=0.478/0.354 (balanced)

Ye poori tarah, ek dramatic reversal hai — aur iski wajah simple hai: purana 'significant' summer-window result, ek incomplete sample (154/251, Sikkim poori tarah missing) par bana tha. Jab Sikkim wapas aaya, poora, complete, same-day sample par test dobara chala, aur wahi signal jo incomplete data mein dikha raha tha, gayab ho gaya. Ye is poore project ka sabse clear reminder hai: ek 'significant' result sirf tab tak trust karne layak hai, jab tak uske peeche ka data poora aur consistent ho — aur incomplete data, chahe pehli nazar mein kitna bhi convincing lage, ek false signal bana sakta hai.

Recap — Is PDF Mein Kya Seekha

Is PDF Mein Kya Seekha: (1) `extract_satellite_data.py` har gaon ko 500m buffer karta hai, Sentinel-2 se NDBI (SWIR1-NIR)/(SWIR1+NIR) aur VIIRS se night-lights nikaalta hai, dono full_year aur summer windows mein — koi cloud-free image na mile toh null, zero nahi. (2) `mask_s2_clouds()` QA60 bitmask se cloud/cirrus exclude karta hai — ye original method hai, ek refined SCL-based method PDF 9 mein aayega. (3) `extract_control_satellite_data.py` bilkul wahi logic reuse karta hai, sirf resume-strategy alag hai — coordinate-based, kyunki control-list har baar regenerate hoti hai (Entry 15). (4) Entry 22 mein do real bugs mile (lat/lon-missing checkpoint, aur `Series.round` vs scalar round ka mismatch) jinki wajah se poori, fresh, same-day re-extraction hui. (5) Us re-extraction ne is poore project ka sabse bada reversal diya — jo 'significant, surviving' result the, sab null ho gaye, kyunki purana result ek incomplete (Sikkim-missing) sample par bana tha.

Ye raw NDBI aur night-lights numbers hi hain jo aage PDF 11 (H1 test — treated villages apna before/after kaise compare karte hain) aur PDF 12 (H4 DiD test — treated vs control comparison) mein statistically test honge. Bina in do extraction-files ke, koi bhi statistical test ke paas test karne ke liye kuch hota hi nahi.

Agla step: PDF 9 — Cloud Masking (`extract_scl_cloud_mask_ndbi.py`, `scl_vs_qa60_comparison.py`, SCL snow-bug story ke saath), jab bhi Sakshi 'next' bole.